

Эмуляция и измерение потерь пакетов в глобальных сетях

Лабораторная работа № 5

Шулуужук Айраана НПИбд-02-22

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
2.1	Запуск лабораторной топологии	6
2.2	Добавление потери пакетов на интерфейс, подключённый к эмулируемой глобальной сети	8
2.3	Добавление значения корреляции для потери пакетов в эмулируемой глобальной сети	9
2.4	Добавление повреждения пакетов в эмулируемой глобальной сети	10
2.5	Добавление переупорядочивания пакетов в интерфейс подключения к эмулируемой глобальной сети	11
2.6	Добавление дублирования пакетов в интерфейс подключения к эмулируемой глобальной сети	12
2.7	Воспроизведение экспериментов. Добавление потери пакетов на интерфейс, подключённый к эмулируемой глобальной сети	14
2.8	Задание для самостоятельной работы	15
3	Выводы	20

Список иллюстраций

2.1	запуск виртуальной машины и исправление прав доступа .	6
2.2	запуск графических окон	7
2.3	проверка подключения между хостами	7
2.4	проверка подключения соединения с добавлением 10% потерь пакетов	8
2.5	эмуляция глобальной сети с потерей пакетов в обоих направлениях	9
2.6	добавление на интерфейсе узла h1 коэффициента потери пакетов 50% и проверка соединения	10
2.7	проверка конфигурации с помощью инструмента iPerf3 для проверки повторных передач	10
2.8	запуск iPerf3 в режиме сервера в терминале хоста h2	11
2.9	добавление на интерфейсе узла h1 правила	11
2.10	проверка соединения	12
2.11	добавление для интерфейса узла h1 зададим правило с дублированием 50% пакетов	13
2.12	проверка соединения	13
2.13	скрипт для эксперимента lab_netem_ii.py	14
2.14	скрипт для вывода на экран информации о потерях пакетов	14
2.15	скрипт для управления процессом проведения эксперимента	15
2.16	результаты эксперимента	15
2.17	скрипт для эксперимента lab_netem_ii.py	16
2.18	результат эксперимента	16
2.19	скрипт для эксперимента lab_netem_ii.py	17
2.20	результат эксперимента	17
2.21	скрипт для эксперимента lab_netem_ii.py	18
2.22	результат эксперимента	18
2.23	скрипт для эксперимента lab_netem_ii.py	19
2.24	результат эксперимента	19

Список таблиц

1 Цель работы

Основной целью работы является получение навыков проведения интерактивных экспериментов в среде Mininet по исследованию параметров сети, связанных с потерей, дублированием, изменением порядка и повреждением пакетов при передаче данных. Эти параметры влияют на производительность протоколов и сетей.

2 Выполнение лабораторной работы

2.1 Запуск лабораторной топологии

Запустим виртуальную среду с mininet. Из основной ОС подключимся к виртуальной машине. В виртуальной машине mininet при необходимости исправим права запуска X-соединения. Скопируйте значение куки (MIT magic cookie)¹ своего пользователя mininet в файл для пользователя root (рис. 2.1)

```
mininet@mininet-vm:~$ xauth list $DISPLAY
mininet-vm/unix:10 MIT-MAGIC-COOKIE-1 fa5520b19462a53b131d09eb09dd97c6
mininet@mininet-vm:~$ sudo -i
root@mininet-vm:~# xauth add mininet-vm/unix:10 MIT-MAGIC-COOKIE-1 fa5520b1946
2a53b131d09eb09dd97c6
root@mininet-vm:~# logout
mininet@mininet-vm:~$
```

Рис. 2.1: запуск виртуальной машины и исправление прав доступа

Зададим простейшую топологию, состоящую из двух хостов и коммутатора с назначенной по умолчанию mininet сетью 10.0.0.0/8. После введения этой команды запустятся терминалы двух хостов, коммутатора и контроллера. Терминалы коммутатора и контроллера можно закрыть (рис. 2.2)



Рис. 2.2: запуск графических окон

Проверим подключение между хостами h1 и h2 с помощью команды ping с параметром -c 6 (рис. 2.3)

min/avg/max/mdev = 0.052/0.569/2.820/1.010 ms

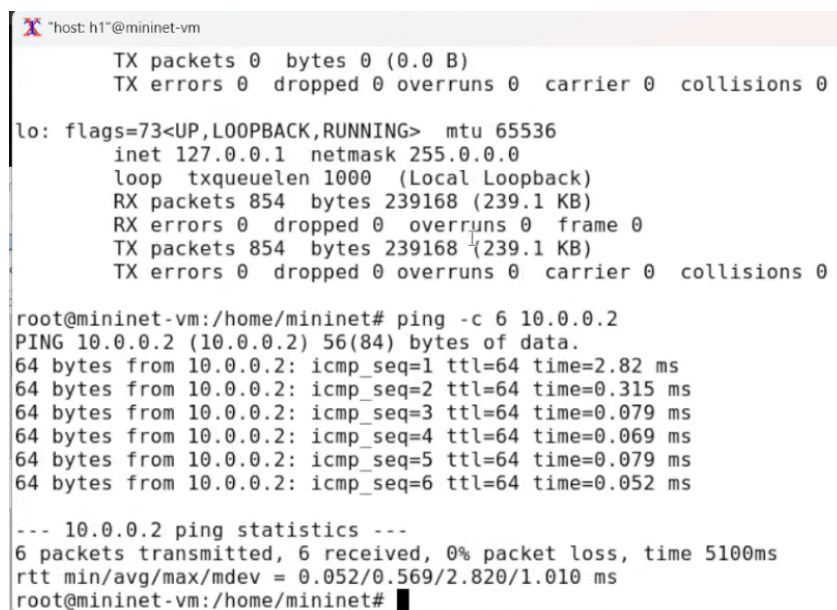


Рис. 2.3: проверка подключения между хостами

2.2 Добавление потери пакетов на интерфейс, подключённый к эмулируемой глобальной сети

Пакеты могут быть потеряны в процессе передачи из-за таких факторов, как битовые ошибки и перегрузка сети. Скорость потери данных часто измеряется как процентная доля потерянных пакетов по отношению к количеству отправленных пакетов

На хосте h1 добавим 10% потерь пакетов к интерфейсу h1-eth0. Проверим, что на соединении от хоста h1 к хосту h2 имеются потери пакетов, используя команду `ping` с параметром `-c 100` с хоста h1. Параметр `-c` указывает общее количество пакетов для отправки. Обратите внимание на значения `icmp_seq`. Некоторые номера последовательности отсутствуют из-за потери пакетов. Процент потерянных пакетов составил 5% (рис. 2.4)

```
root@mininet-vm:/home/mininet# sudo tc qdisc add dev h1-eth0 root netem loss 10
%
root@mininet-vm:/home/mininet# ping -c 100 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.446 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.393 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.203 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.077 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.053 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=0.062 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=0.074 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=0.050 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=0.073 ms
64 bytes from 10.0.0.2: icmp_seq=11 ttl=64 time=0.053 ms
64 bytes from 10.0.0.2: icmp_seq=12 ttl=64 time=0.048 ms
64 bytes from 10.0.0.2: icmp_seq=13 ttl=64 time=0.041 ms
64 bytes from 10.0.0.2: icmp_seq=14 ttl=64 time=0.064 ms
64 bytes from 10.0.0.2: icmp_seq=15 ttl=64 time=0.057 ms
64 bytes from 10.0.0.2: icmp_seq=16 ttl=64 time=0.050 ms
64 bytes from 10.0.0.2: icmp_seq=17 ttl=64 time=0.054 ms
64 bytes from 10.0.0.2: icmp_seq=18 ttl=64 time=0.063 ms
■
```

Рис. 2.4: проверка подключения соединения с добавлением 10% потерь пакетов

Для эмуляции глобальной сети с потерей пакетов в обоих направлениях необходимо к соответствующему интерфейсу на хосте h2 также добавить 10% потерь пакетов. Проверьте, что соединение между хостом h1 и хостом h2 имеет больший процент потерянных данных (10% от хоста h1 к хосту h2 и 10% от хоста h2 к хосту h1), повторив команду `ping` с параметром `-c`

100 на терминале хоста h1. Процент потерянных пакетов составило 12% (рис. 2.5)

```
root@mininet-vm:/home/mininet# sudo tc qdisc add dev h2-eth0 root netem loss 10
%
root@mininet-vm:/home/mininet# ping -c 100 10.0.0.1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data:
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=0.646 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=0.361 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=0.049 ms
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=0.078 ms
64 bytes from 10.0.0.1: icmp_seq=5 ttl=64 time=0.065 ms
64 bytes from 10.0.0.1: icmp_seq=6 ttl=64 time=0.065 ms
64 bytes from 10.0.0.1: icmp_seq=7 ttl=64 time=0.051 ms
64 bytes from 10.0.0.1: icmp_seq=8 ttl=64 time=0.063 ms
64 bytes from 10.0.0.1: icmp_seq=9 ttl=64 time=0.060 ms
64 bytes from 10.0.0.1: icmp_seq=10 ttl=64 time=0.047 ms
64 bytes from 10.0.0.1: icmp_seq=11 ttl=64 time=0.049 ms
64 bytes from 10.0.0.1: icmp_seq=12 ttl=64 time=0.051 ms
64 bytes from 10.0.0.1: icmp_seq=14 ttl=64 time=0.051 ms
```

Рис. 2.5: эмуляция глобальной сети с потерей пакетов в обоих направлениях

2.3 Добавление значения корреляции для потери пакетов в эмулируемой глобальной сети

Добавим на интерфейсе узла h1 коэффициент потери пакетов 50% (такой высокий уровень потери пакетов маловероятен), и каждая последующая вероятность зависит на 50% от последней. Проверим, что на соединении от хоста h1 к хосту h2 имеются потери пакетов, используя команду ping с параметром -c 50 с хоста h1. Процент потерянных пакетов составило 66 %. Номера пакетов: 1 2 4 6 7 8 9 10 12 13 17 18 19 20 21 22 23 24 25 26 27 31 32 33 34 35 38 39 40 41 46 47 48 (рис. 2.6)

```

X "host: h1" @mininet-vm
root@mininet-vm:/home/mininet# ping -c 50 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.572 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.318 ms
64 bytes from 10.0.0.2: icmp_seq=11 ttl=64 time=2048 ms
64 bytes from 10.0.0.2: icmp_seq=14 ttl=64 time=0.063 ms
64 bytes from 10.0.0.2: icmp_seq=15 ttl=64 time=0.064 ms
64 bytes from 10.0.0.2: icmp_seq=16 ttl=64 time=0.048 ms
64 bytes from 10.0.0.2: icmp_seq=28 ttl=64 time=0.174 ms
64 bytes from 10.0.0.2: icmp_seq=29 ttl=64 time=0.065 ms
64 bytes from 10.0.0.2: icmp_seq=30 ttl=64 time=0.045 ms
64 bytes from 10.0.0.2: icmp_seq=36 ttl=64 time=0.061 ms
64 bytes from 10.0.0.2: icmp_seq=37 ttl=64 time=0.059 ms
64 bytes from 10.0.0.2: icmp_seq=42 ttl=64 time=0.062 ms
64 bytes from 10.0.0.2: icmp_seq=43 ttl=64 time=0.078 ms
64 bytes from 10.0.0.2: icmp_seq=44 ttl=64 time=0.061 ms
64 bytes from 10.0.0.2: icmp_seq=45 ttl=64 time=0.050 ms
64 bytes from 10.0.0.2: icmp_seq=49 ttl=64 time=0.062 ms
64 bytes from 10.0.0.2: icmp_seq=50 ttl=64 time=0.063 ms

--- 10.0.0.2 ping statistics ---
50 packets transmitted, 17 received, 66% packet loss, time 50169ms
rtt min/avg/max/mdev = 0.045/120.605/2048.456/481.962 ms, pipe 3
root@mininet-vm:/home/mininet# █

```

Рис. 2.6: добавление на интерфейсе узла h1 коэффициента потери пакетов 50% и проверка соединения

2.4 Добавление повреждения пакетов в эмулируемой глобальной сети

Добавим на интерфейсе узла h1 0,01% повреждения пакетов (рис. 2.7) (рис. 2.8)

```

root@mininet-vm:/home/mininet# sudo tc qdisc add dev h1-eth0 root netem corrupt
0.01%
root@mininet-vm:/home/mininet# iperf3 -c 10.0.0.2
Connecting to host 10.0.0.2, port 5201
[ 7] local 10.0.0.1 port 43940 connected to 10.0.0.2 port 5201
[ ID] Interval            Transfer          Bitrate          Retr  Cwnd
[ 7] 0.00-1.00 sec        4.20 GBytes      36.0 Gbits/sec   10    773 KBytes
[ 7] 1.00-2.00 sec        4.13 GBytes      35.5 Gbits/sec   15    1.77 MBytes
[ 7] 2.00-3.00 sec        4.40 GBytes      37.8 Gbits/sec    7    2.25 MBytes
[ 7] 3.00-4.00 sec        4.36 GBytes      37.4 Gbits/sec    9    945 KBytes
[ 7] 4.00-5.00 sec        3.96 GBytes      34.0 Gbits/sec    5    1.58 MBytes
[ 7] 5.00-6.00 sec        3.96 GBytes      33.9 Gbits/sec   12    1.60 MBytes
[ 7] 6.00-7.00 sec        4.22 GBytes      36.3 Gbits/sec    8    1.72 MBytes
[ 7] 7.00-8.00 sec        4.08 GBytes      35.0 Gbits/sec   11    1.83 MBytes
[ 7] 8.00-9.00 sec        3.99 GBytes      34.3 Gbits/sec    7    2.22 MBytes
[ 7] 9.00-10.00 sec       4.10 GBytes      35.2 Gbits/sec   10    977 KBytes
- - - - -
[ ID] Interval            Transfer          Bitrate          Retr
[ 7] 0.00-10.00 sec       41.4 GBytes      35.6 Gbits/sec   94
[ 7] 0.00-10.00 sec       41.4 GBytes      35.5 Gbits/sec
iperf Done.
root@mininet-vm:/home/mininet# █

```

Рис. 2.7: проверка конфигурации с помощью инструмента iPerf3 для проверки повторных передач

```

host: h2" @mininet-vm
warning: this system does not seem to support IPv6 - trying IPv4
-----
Server listening on 5201
-----
Accepted connection from 10.0.0.1, port 43938
[ 7] local 10.0.0.2 port 5201 connected to 10.0.0.1 port 43940
[ ID] Interval      Transfer    Bitrate
[ 7] 0.00-1.00 sec  4.19 GBytes 36.0 Gbits/sec
[ 7] 1.00-2.00 sec  4.13 GBytes 35.4 Gbits/sec
[ 7] 2.00-3.00 sec  4.40 GBytes 37.8 Gbits/sec
[ 7] 3.00-4.00 sec  4.36 GBytes 37.4 Gbits/sec
[ 7] 4.00-5.00 sec  3.96 GBytes 34.0 Gbits/sec
[ 7] 5.00-6.00 sec  3.95 GBytes 33.9 Gbits/sec
[ 7] 6.00-7.00 sec  4.22 GBytes 36.3 Gbits/sec
[ 7] 7.00-8.00 sec  4.08 GBytes 35.1 Gbits/sec
[ 7] 8.00-9.00 sec  3.98 GBytes 34.2 Gbits/sec
[ 7] 9.00-10.00 sec 4.10 GBytes 35.2 Gbits/sec
-----
[ ID] Interval      Transfer    Bitrate
[ 7] 0.00-10.00 sec 41.4 GBytes 35.5 Gbits/sec
-----
Server listening on 5201
-----
receiver

```

Рис. 2.8: запуск iPerf3 в режиме сервера в терминале хоста h2

2.5 Добавление переупорядочивания пакетов в интерфейс подключения к эмулируемой глобальной сети

Добавим на интерфейсе узла h1 правило: 25% пакетов (со значением корреляции 50%) будут отправлены немедленно, а остальные 75% будут задержаны на 10 мс (рис. 2.9) (рис. 2.10)

```

host: h1" @mininet-vm
root@mininet-vm:/home/mininet# sudo tc qdisc add dev h1-eth0 root netem delay 10ms reorder 25% 50%
root@mininet-vm:/home/mininet# ping -c 20 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=11.3 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=10.5 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=11.1 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=10.5 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=10.9 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=10.3 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=10.1 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=10.6 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=10.9 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=11.0 ms
64 bytes from 10.0.0.2: icmp_seq=11 ttl=64 time=10.5 ms
64 bytes from 10.0.0.2: icmp_seq=12 ttl=64 time=10.4 ms
64 bytes from 10.0.0.2: icmp_seq=13 ttl=64 time=10.8 ms
64 bytes from 10.0.0.2: icmp_seq=14 ttl=64 time=10.1 ms
64 bytes from 10.0.0.2: icmp_seq=15 ttl=64 time=10.2 ms
64 bytes from 10.0.0.2: icmp_seq=16 ttl=64 time=10.1 ms
64 bytes from 10.0.0.2: icmp_seq=17 ttl=64 time=10.8 ms
64 bytes from 10.0.0.2: icmp_seq=18 ttl=64 time=11.0 ms
64 bytes from 10.0.0.2: icmp_seq=19 ttl=64 time=10.7 ms

```

Рис. 2.9: добавление на интерфейсе узла h1 правила

```

X "host: h1" @mininet-vm
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=10.1 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=10.6 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=10.1 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=10.3 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=10.8 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=11.2 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=10.9 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=10.1 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=11.4 ms
64 bytes from 10.0.0.2: icmp_seq=11 ttl=64 time=10.6 ms
64 bytes from 10.0.0.2: icmp_seq=12 ttl=64 time=10.2 ms
64 bytes from 10.0.0.2: icmp_seq=13 ttl=64 time=10.1 ms
64 bytes from 10.0.0.2: icmp_seq=14 ttl=64 time=10.1 ms
64 bytes from 10.0.0.2: icmp_seq=15 ttl=64 time=10.5 ms
64 bytes from 10.0.0.2: icmp_seq=16 ttl=64 time=0.073 ms
64 bytes from 10.0.0.2: icmp_seq=17 ttl=64 time=10.1 ms
64 bytes from 10.0.0.2: icmp_seq=18 ttl=64 time=10.7 ms
64 bytes from 10.0.0.2: icmp_seq=19 ttl=64 time=10.1 ms
64 bytes from 10.0.0.2: icmp_seq=20 ttl=64 time=10.1 ms

--- 10.0.0.2 ping statistics ---
20 packets transmitted, 20 received, 0% packet loss, time 19056ms
rtt min/avg/max/mdev = 0.073/9.924/11.359/2.292 ms
root@mininet-vm:/home/mininet#

```

Рис. 2.10: проверка соединения

2.6 Добавление дублирования пакетов в интерфейс подключения к эмулируемой глобальной сети

Для интерфейса узла h1 зададим правило с дублированием 50% пакетов (т.е. 50% пакетов должны быть получены дважды) (рис. 2.11)

```

root@mininet-vm:/home/mininet# sudo tc qdisc add dev h1-eth0 root netem duplica
te 50%
root@mininet-vm:/home/mininet# ping -c 20 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=1.10 ms
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=1.13 ms (DUP!)
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.513 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=1.29 ms (DUP!)
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.230 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.587 ms (DUP!)
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.079 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.080 ms (DUP!)
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.067 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.049 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=0.050 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=0.062 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=0.084 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=0.049 ms
64 bytes from 10.0.0.2: icmp_seq=11 ttl=64 time=0.053 ms
64 bytes from 10.0.0.2: icmp_seq=11 ttl=64 time=0.053 ms (DUP!)
64 bytes from 10.0.0.2: icmp_seq=12 ttl=64 time=0.064 ms
64 bytes from 10.0.0.2: icmp_seq=13 ttl=64 time=0.052 ms
64 bytes from 10.0.0.2: icmp_seq=13 ttl=64 time=0.052 ms (DUP!)

```

Рис. 2.11: добавление для интерфейса узла h1 зададим правило с дублированием 50% пакетов

Проверим, что на соединении от хоста h1 к хосту h2 имеются дублированные пакеты, используя команду ping с параметром -c 20 с хоста h1. Дубликаты пакетов помечаются как DUP! (рис. 2.12)

```

64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=0.050 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=0.062 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=0.084 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=0.049 ms
64 bytes from 10.0.0.2: icmp_seq=11 ttl=64 time=0.053 ms
64 bytes from 10.0.0.2: icmp_seq=11 ttl=64 time=0.053 ms (DUP!)
64 bytes from 10.0.0.2: icmp_seq=12 ttl=64 time=0.064 ms
64 bytes from 10.0.0.2: icmp_seq=13 ttl=64 time=0.052 ms
64 bytes from 10.0.0.2: icmp_seq=13 ttl=64 time=0.052 ms (DUP!)
64 bytes from 10.0.0.2: icmp_seq=14 ttl=64 time=0.050 ms
64 bytes from 10.0.0.2: icmp_seq=14 ttl=64 time=0.050 ms (DUP!)
64 bytes from 10.0.0.2: icmp_seq=15 ttl=64 time=0.048 ms
64 bytes from 10.0.0.2: icmp_seq=16 ttl=64 time=0.057 ms
64 bytes from 10.0.0.2: icmp_seq=17 ttl=64 time=0.048 ms
64 bytes from 10.0.0.2: icmp_seq=17 ttl=64 time=0.048 ms (DUP!)
64 bytes from 10.0.0.2: icmp_seq=18 ttl=64 time=0.064 ms
64 bytes from 10.0.0.2: icmp_seq=18 ttl=64 time=0.064 ms (DUP!)
64 bytes from 10.0.0.2: icmp_seq=19 ttl=64 time=0.046 ms
64 bytes from 10.0.0.2: icmp_seq=20 ttl=64 time=0.085 ms

--- 10.0.0.2 ping statistics ---
20 packets transmitted, 20 received, +9 duplicates, 0% packet loss, time 19408ms
rtt min/avg/max/mdev = 0.046/0.213/1.289/0.350 ms
root@mininet-vm:/home/mininet#

```

Рис. 2.12: проверка соединения

2.7 Воспроизведение экспериментов. Добавление потери пакетов на интерфейс, подключённый к эмулируемой глобальной сети

Создадим скрипт для эксперимента lab_netem_ii.py (рис. 2.13)

```
/home/mini-tem ii.py [-M--] 14 L: [ 23+ 0 23/ 49] *(495 /1134b) 105 0x069 [*][X]
info( "Add switch\n" )
s1 = net.addSwitch( "s1" )

info( "Creating links\n" )
net.addLink( h1, s1 )
net.addLink( h2, s1 )

info( "Starting network" )
net.start()

info( "Set delay" )
h1.cmdPrint( "tc qdisc add dev h1-eth0 root netem loss 10% " )
h2.cmdPrint( "tc qdisc add dev h2-eth0 root netem loss 10% " )

time.sleep(10)

info( "Ping" )
h1.cmdPrint( "ping -c 100", h2.IP(), "[ -qmp 'timed' ] awk '{print $5, $7}'" )

info( "Stopping network" )
net.stop()

if __name__ == '__main__':
    setLogLevel( 'info' )
    emptyNet()
```

Рис. 2.13: скрипт для эксперимента lab_netem_ii.py

Создадим скрипт, который на экран или в отдельный файл выводит информацию о потерях пакетов (рис. 2.14)

```
/home/mini-op/lost p [----] 57 L: [ 1+14 15/ 21] *(593 / 705b) 10 0x00A [*][
def lost_p(file_path='ping.dat', total_packets=100):
    with open(file_path, 'r') as f:
        <----->lines = f.readlines()
        <----->received_packets = set()
        <----->for line in lines:
            <----->packet_number = int(line.split()[0])
            <----->received_packets.add(packet_number)

    if total_packets > 0:
        <----->lost_packets = set(range(1, total_packets + 1)) - received_packets
        <----->lost_packet_count = len(lost_packets)
        <----->loss_percentage = (lost_packet_count / total_packets) * 100
        <----->print(f'Lost packets: {lost_packet_count}')
        <----->print(f'Lost packet numbers: {sorted(list(lost_packets))}')
        <----->print(f'Lost percentage: {loss_percentage:.2f}%')
    else:
        <----->print("Total packets should be more than 0")

if __name__ == '__main__':
    analyze_ping_results()
```

Рис. 2.14: скрипт для вывода на экран информации о потерях пакетов

Создадим Makefile для управления процессом проведения эксперимента (рис. 2.15)

```
/home/mini~/Makefile [-M--] 13 L:[ 1+ 0 1/ 11
all: ping.dat

ping.dat:
<----->sudo python lab_netem_ii.py
<----->sudo chown mininet:mininet ping.dat

stats:
<----->sudo python lost_p.py

clean:
<----->-rm -f *.dat
```

Рис. 2.15: скрипт для управления процессом проведения эксперимента

Выполним эксперимент (рис. 2.16)

```
mininet@mininet-vm:~/work/lab_netem_ii/simple-drop$ make stats
sudo python lost_p.py
Lost packets: 28
Lost packet numbers: [1, 2, 3, 12, 20, 21, 22, 23, 28, 33, 37, 38, 39, 40, 41, 42,
44, 58, 60, 66, 69, 73, 74, 81, 87, 89, 92, 96]
Lost percentage: 28.00%
mininet@mininet-vm:~/work/lab_netem_ii/simple-drop$ make clean
rm -f *.dat
```

Рис. 2.16: результаты эксперимента

2.8 Задание для самостоятельной работы

Самостоятельно реализуйте воспроизводимые эксперименты по исследованию параметров сети, связанных с потерей, изменением порядка и повреждением пакетов при передаче данных.

- Добавление значения корреляции для потери пакетов в эмулируемой глобальной сети (рис. 2.17)

```
/home/mininet-ii.py [BM--] 0 L:[ 24+17 41/ 49] *(1016/1137b) 10 0x00A [*][X]
s1 = net.addSwitch( 's1' )

info( 'Creating links\n' )
net.addLink( h1, s1 )
net.addLink( h2, s1 )

info( 'Starting network\n' )
net.start()

info( 'Set delays\n' )
h1.cmdPrint( 'tc qdisc add dev h1-eth0 root netem loss 50% 50%' )
h2.cmdPrint( 'tc qdisc add dev h2-eth0 root netem loss 10%' )

time.sleep(10)

info( 'Ping\n' )
h1.cmdPrint( 'ping -c 50', h2.IP(), '| grep "time=" | awk \'{print $5, $7}\'' )

info( 'Stopping network\n' )
net.stop()

if __name__ == '__main__':
    setLogLevel( 'info' )
    emptyNet()
```

Рис. 2.17: скрипт для эксперимента lab_netem_ii.py

Выполним эксперимент (рис. 2.18)

```
mininet@mininet-vm:~/work/lab_netem_ii/correlation-drop$ make stats
sudo python lost_p.py
Lost packets: 56
Lost packet numbers: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 12, 13, 16, 17, 18, 19, 24, 25, 26, 27, 29, 32, 35, 38, 39, 40, 42, 46, 47, 48, 51, 54, 59, 60, 61, 62, 63, 65, 67, 68, 69, 71, 75, 76, 77, 79, 80, 81, 91, 92, 93, 95, 96, 97, 98, 100]
Lost percentage: 56.00%
mininet@mininet-vm:~/work/lab_netem_ii/correlation-drop$ make clean
rm -f *.dat
mininet@mininet-vm:~/work/lab_netem_ii/correlation-drop$
```

Рис. 2.18: результат эксперимента

- Добавление повреждения пакетов в эмулируемой глобальной сети (рис. 2.19)


```

/home/mininet ii.py [B---] 18 L: [ 23+14 37/ 49] *(855 /1139b) 10 0x00A [*][X]
info( 'Adding switch\n' )
s1 = net.addSwitch( 's1' )

info( 'Creating links\n' )
net.addLink( h1, s1 )
net.addLink( h2, s1 )

info( 'Starting network\n' )
net.start()

info( 'Set delay\n' )
h1.cmdPrint( 'tc qdisc add dev h1-eth0 root netem corrupt 0.01%' )
h2.cmdPrint( 'tc qdisc add dev h2-eth0 root netem loss 10%' )

time.sleep(10)

info( 'Ping\n' )
h1.cmdPrint( 'ping -c 100', h2.IP(), '[ -q -p "times" ] awk \'{print $5, $7}\'' )

info( 'Stopping network\n' )
net.stop()

if __name__ == '__main__':
    setLogLevel( 'info' )
    emptyNet()

```

1Help 2Save 3Mark 4Replac 5Copy 6Move 7Search 8Delete 9PullDn 10Quit

Рис. 2.19: скрипт для эксперимента lab_netem_ii.py

Выполним эксперимент (рис. 2.20)

```

mininet@mininet-vm:~/work/lab_netem_ii/damage-drop$ make stats
sudo python lost_p.py
Lost packets: 10
Lost packet numbers: [7, 19, 24, 27, 32, 34, 51, 78, 89, 90]
Lost percentage: 10.00%
mininet@mininet-vm:~/work/lab_netem_ii/damage-drop$

```

Рис. 2.20: результат эксперимента

- Добавление переупорядочивания пакетов в интерфейс подключения к эмулируемой глобальной сети (рис. 2.21)

```

/home/mininet ii.py [BM--] 55 L:[ 24+16 40/ 49] *(944 /1152b) 34 0x022 [*][X]
s1 = net.addSwitch( 's1' )

info( 'Creating Links\n' )
net.addLink( h1, s1 )
net.addLink( h2, s1 )

info( 'Starting network' )
net.start()

info( 'Set delay' )
h1.cmdPrint( 'tc qdisc add dev h1-eth0 root netem delay 10ms reorder 25% 50%' )
h2.cmdPrint( 'tc qdisc add dev h2-eth0 root netem loss 10%' )

time.sleep(10)

info( 'Ping' )
h1.cmdPrint( 'ping -c 100', h2.IP(), '| grep "time=" | awk '{print $2, $3}' )

info( 'Stopping network' )
net.stop()

if __name__ == '__main__':
    setLogLevel( 'info' )
    emptyNet()

```

Рис. 2.21: скрипт для эксперимента lab_netem_ii.py

Выполним эксперимент (рис. 2.22)

```

mininet@mininet-vm:~/work/lab_netem_ii/reorder-drop$ make stats
sudo python lost_p.py
Lost packets: 15
Lost packet numbers: [1, 5, 9, 10, 17, 33, 36, 57, 59, 60, 64, 71, 79, 80, 88]
Lost percentage: 15.00%
mininet@mininet-vm:~/work/lab_netem_ii/reorder-drop$

```

Рис. 2.22: результат эксперимента

- Добавление дублирования пакетов в интерфейс подключения к эмулируемой глобальной сети (рис. 2.23)

```
/home/mininet~ ii.py [-M--] 60 L:[ 24+11 35/ 49] *(830 /1138b) 48 0x030 [*][X]
s1 = net.addSwitch( 's1' )

info( 'Creating links\n' )
net.addLink( h1, s1 )
net.addLink( h2, s1 )

info( 'Starting network' )
net.start()

info( 'Net delay' )
h1.cmdPrint( 'tc qdisc add dev h1-eth0 root netem duplicate 50%' )
h2.cmdPrint( 'tc qdisc add dev h2-eth0 root netem loss 0' )

time.sleep(10)

info( 'Ping' )
h1.cmdPrint( 'ping -c 20', h2.IP(), '-i 0.001 -s 64 -P 1' )

info( 'Stopping network' )
net.stop()

if __name__ == '__main__':
    setLogLevel( 'info' )
    emptyNet()
```

Рис. 2.23: скрипт для эксперимента lab_netem_ii.py

Выполним эксперимент (рис. 2.24)

```
mininet@mininet-vm:~/work/lab_netem_ii/duplicate-drop$ make stats
sudo python lost_p.py
Lost packets: 83
Lost packet numbers: [9, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33,
34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54,
55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74,
75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95,
96, 97, 98, 99, 100]
Lost percentage: 83.00%
mininet@mininet-vm:~/work/lab_netem_ii/duplicate-drop$ make stats
```

Рис. 2.24: результат эксперимента

3 Выводы

В результате выполнения лабораторной работы были получены навыки проведения интер-активных экспериментов в среде Mininet по исследованию параметров сети, связанных с потерей, дублированием, изменением порядка и повреждением пакетов при передаче данных. Эти параметры влияют на производительность протоколов и сетей.